



Sistemas Distribuídos com **Mensageria** (fila, tópico, DLQ)

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Módulo: 05 · Sistemas Distribuídos, Mensageria e Eventos

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

filas / RabbitMQ / Kafka

1. Contexto

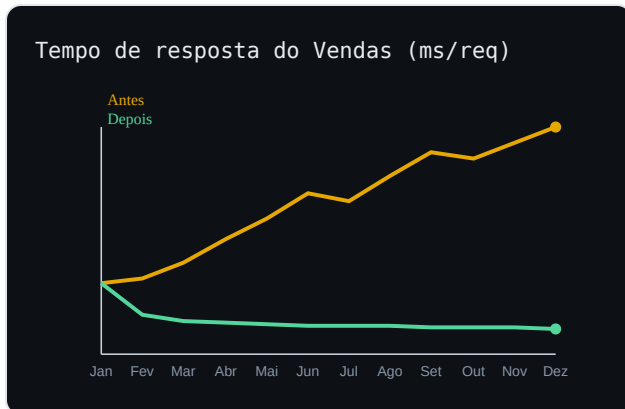
A FV precisa que o time de **Vendas** registre um negócio e, em segundos, o time de **Financeiro**, o de **Marketing** e o de **Operação** reajam: sem que um sistema espere o outro travar. Hoje tudo é feito por chamada síncrona: o Vendas chama o Financeiro, que chama o Marketing... se um cai, a cadeia inteira falha. É o chamado **acoplamento forte**.

ANALOGIA DO RESTAURANTE

Imagine um restaurante onde o garçom só libera o pedido se ele mesmo levar o prato à cozinha, encher o copo, cobrar no caixa e lavar a louça antes de atender a próxima mesa. Ninguém come. A **mensageria** é o *comanda na cozinha*: o garçom anota o pedido (evento) e segue. A cozinha, o caixa e a lavanderia lêem a comanda no próprio ritmo. Cada setor trabalha independente.

2. Diagnóstico

Com chamadas síncronas, uma lentidão no Financeiro trava o Vendas. Picos de campanha geram **fila invisível** de requisições perdidas. Não há replay: se o Marketing estava fora do ar, o evento sumiu. O custo é operacional (timeout em cascata) e de oportunidade (lead esquecido).

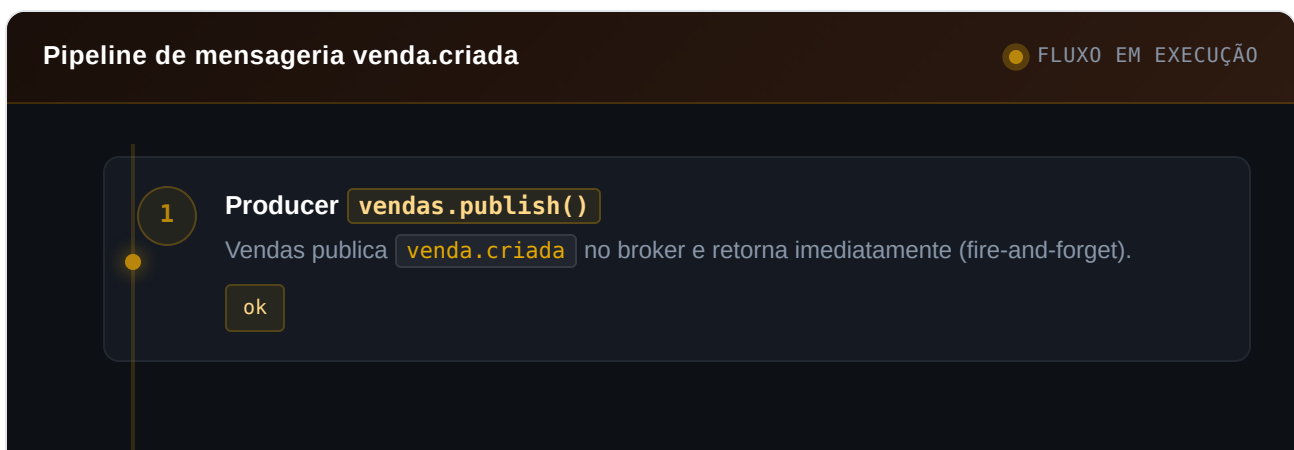


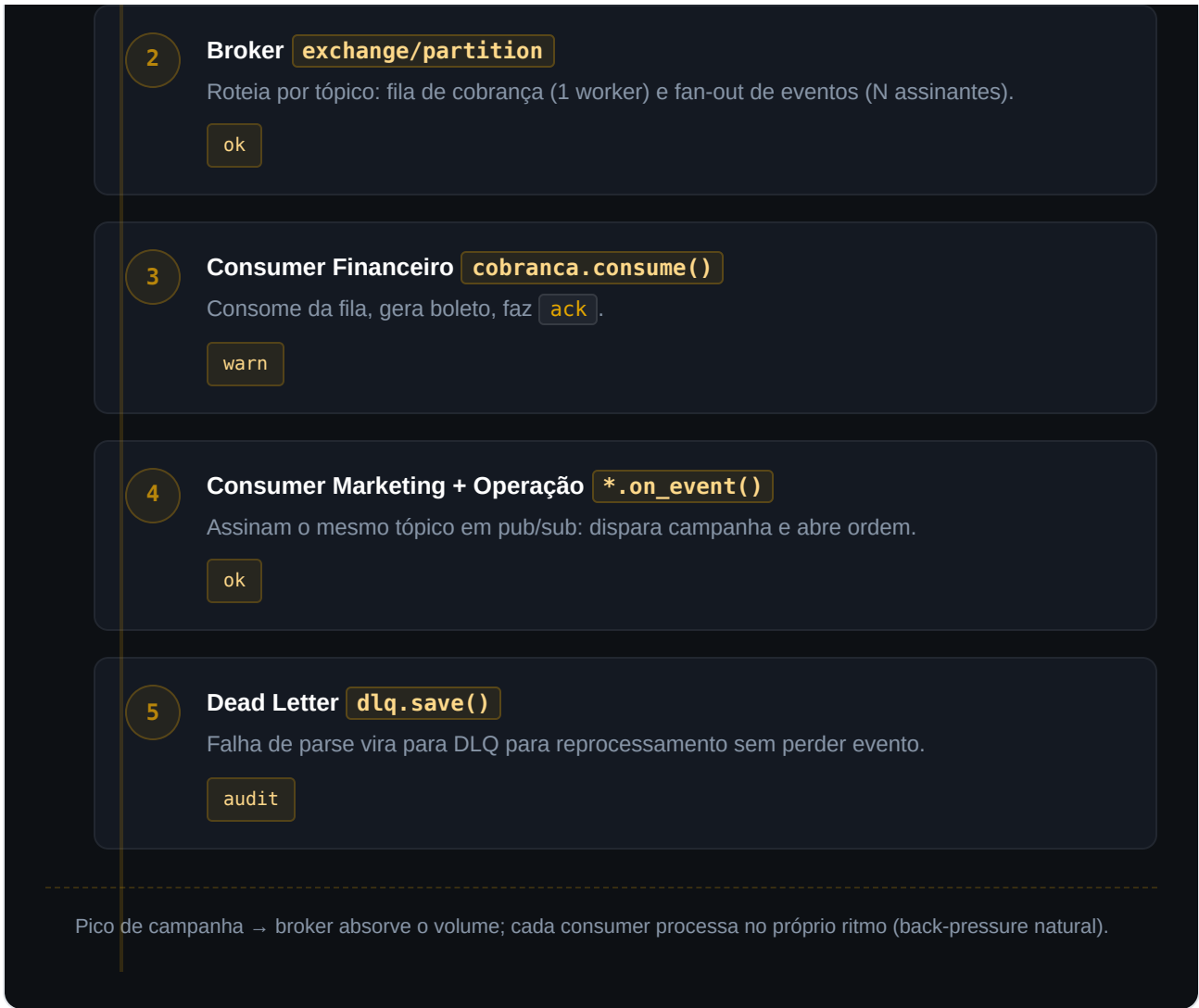
Raiz do problema: o Vendas é *acoplado* ao Financeiro/Marketing via chamada direta. Não existe **desacoplamento** nem **buffer** para absorver picos: o sistema não escala porque cada parte depende do tempo de resposta da outra.

3. Solução

Inserir um **broker de mensageria** (RabbitMQ para filas de trabalho, Kafka para eventos de fluxo) entre os sistemas. O Vendas *publica* um evento `venda.criada` e segue em ~2ms. Cada time *assina* o que precisa. Há dois padrões principais: **fila (queue)**, onde 1 mensagem é consumida por 1 worker (tarefa), e **publish/subscribe**, onde 1 mensagem é entregue a N assinantes (evento difundido).

4. Como funciona (pipeline)





5. Antes vs Depois

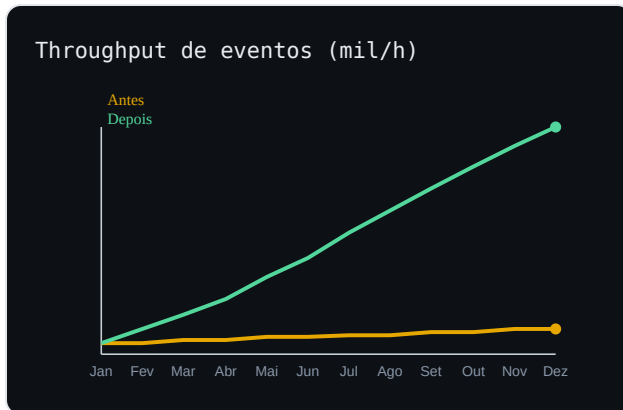
CENÁRIO	ANTES (SÍNCRONO)	DEPOIS (MENSAGERIA)
Falha em 1 sistema	Trava a cadeia	Isolado ao consumer
Pico de campanha	Timeouts / perda	Buffer no broker
Replay de evento	Impossível	Replay por offset (Kafka)
Acoplamento	Forte (Vendas → Fin)	Fraco (evento)

6. Entregas

- Diagrama topologia-mensageria.png: fila vs pub/sub com os 3 times.

- **Worker** cobranca_consumer.py: consumer com ack e DLQ.
- **Producer** vendas_publisher.py: publicação assíncrona via RabbitMQ.
- **Doc** runbook-mensageria.md: como reprocessar a DLQ e subir um consumer.

7. Métricas



Meta: 100% dos eventos de venda processados de forma assíncrona, com *zero* perda em pico e P95 de ponta-a-ponta < 5s.

8. Decisões e tradeoffs

- **RabbitMQ para filas de trabalho e Kafka para eventos de fluxo:** a cobrança usa fila com 1 worker por mensagem e o Marketing e a Operação assinam o mesmo tópico em pub/sub. Troca um HTTP simples por um broker que precisa de operação.
- **ACK explícito com prefetch limitado:** o consumer confirma depois de processar e recebe poucas mensagens por vez, então um consumer lento não estoura. Troca vazão por worker por estabilidade.
- **DLQ após N tentativas com runbook de reprocessamento:** mensagem inválida vai para a DLQ em vez de sumir e volta pelo runbook-mensageria.md. Exige rotina de revisita em menos de 24h para a DLQ não virar depósito esquecido.
- **Publicação fire-and-forget do venda_criada:** o Vendas publica e segue em ~2ms sem esperar os outros times. Troca resposta imediata por consistência eventual.

Fio condutor: cada decisão troca acoplamento por operação explícita (broker, ACK, DLQ), e cada custo operacional tem dono e rotina definidos.

9. Impacto no negócio

Com alvo de 200 msg/s e zero perda em pico, o teste de 1k mensagens com o consumer derrubado prova que pico de campanha vira buffer no broker em vez de timeout em cascata. O P95 de ponta a ponta abaixo de 5s mantém Vendas, Financeiro e Marketing reagindo em segundos, o que reduz lead esquecido e retrabalho de conciliação. O risco passa a ser operacional e conhecido: manter a DLQ revisitada em menos de 24h.

10. Referências de estudo

- **Curso:** "Apache Kafka Séries: Learn Apache Kafka for Beginners" (Udemy, Stephane Maarek).
- **Vídeo:** "RabbitMQ in 100 Seconds" (YouTube, Fireship).
- **Doc oficial:** RabbitMQ Documentation (rabbitmq.com).
- **Doc oficial:** Apache Kafka Documentation (kafka.apache.org).

11. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.